CareerNet — Sprint 2 AI Usage Log

Student: Krish Patel
Course: SOEN 341 — Software Process
Project: CareerNet
Sprint: Sprint 2

AI Usage Log Entry 1 — October 9, 2026

Task ID/Title: KR-1 — Git Branch Setup and Conflict Resolution

Purpose of AI Use:

Used ChatGPT to understand the correct Git workflow for starting Sprint 2 and to troubleshoot unexpected changes involving Sprint 1 documentation files.

Prompt / Request:

Asked ChatGPT how to create a feature branch, what Git commands to run when starting and finishing a work session, and how to handle an unresolved Git conflict safely.

AI Response and Suggested Content:

ChatGPT explained the difference between git pull, git commit, and git push. It suggested checking the Git status, examining the unresolved files, and resolving the repository state before creating the KR-1 feature branch.

Validation:

Ran the Git commands in the VS Code terminal and confirmed that the feature branch feature/KR-1-profile-page was created successfully.

Decision:

Accepted the Git workflow guidance after checking the terminal results.

Reflection:

This helped me understand why a clean repository is important before creating a new feature branch and how to avoid accidentally including unrelated files.

Responsible Person: Krish Patel

AI Usage Log Entry 2 — October 9, 2026

Task ID/Title: KR-1 — Profile Page Implementation

Purpose of AI Use:

Used ChatGPT for frontend development guidance while implementing the Profile Page using React and TypeScript.

Prompt / Request:

Asked ChatGPT to guide me step by step through creating the Profile Page, mock API functions, validation, and navigation.

AI Response and Suggested Content:

ChatGPT suggested creating:

- src/types/profile.ts
- src/api/profile.ts
- src/pages/ProfilePage.tsx

It also suggested adding a protected route, creating the profile form, implementing Save and Edit functionality, and validating the input fields.

Validation:

Reviewed and added the code to the project in VS Code. Ran the frontend and manually tested profile creation, editing, validation, saving, and navigation.

Decision:

Accepted the implementation guidance and used it to build the KR-1 frontend functionality.

Reflection:

I learned how to separate TypeScript data types, API functions, and React page components. Using mock data allowed me to develop the frontend without waiting for the backend team.

Responsible Person: Krish Patel

AI Usage Log Entry 3 — October 9, 2026

Task ID/Title: KR-1 — Authentication and Testing Troubleshooting

Purpose of AI Use:

Used ChatGPT to investigate why registration and login were not working with the shared database and to find a safe way to test the Profile Page independently.

Prompt / Request:

Asked whether the problem was caused by not installing PostgreSQL or Neon locally and how to test the Profile Page without resolving the database connection issue first.

AI Response and Suggested Content:

ChatGPT explained that Neon is cloud-hosted and does not require a local database download. It identified the existing mock authentication option in the frontend and suggested enabling it temporarily to test the profile functionality.

Validation:

Enabled mock authentication, restarted the frontend, logged in using the mock credentials, and manually tested the Profile Page.

Decision:

Accepted the temporary mock authentication approach for KR-1. Real backend integration remains planned for KR-5.

Reflection:

This helped me understand the difference between the frontend, backend, and cloud database, and why frontend features can be tested independently with mock data.

Responsible Person: Krish Patel

AI Usage Log Entry 4 — October 9, 2026

Task ID/Title: KR-1 — GitHub Pull Request Preparation and Review

Purpose of AI Use:

Used ChatGPT to prepare the Pull Request description, understand how to link the task issue, and review the published changes.

Prompt / Request:

Asked ChatGPT where to paste the Pull Request description and whether the completed KR-1 Pull Request looked correct on GitHub.

AI Response and Suggested Content:

ChatGPT drafted a PR description covering the implemented features, manual testing, mock-data limitations, and the Closes #40 issue reference. It also reviewed the PR metadata and changed files.

Validation:

Created PR #67 on GitHub, confirmed that it targeted main, and verified that the five intended KR-1 files were included. Sayf was requested as reviewer.

Decision:

Accepted the Pull Request structure and submitted the changes for review.

Reflection:

The AI assistance helped me prepare a clear Pull Request that documents the work and testing performed. I also learned how to connect a GitHub issue to a Pull Request.

Responsible Person: Krish Patel

AI Usage Log Entry 5 — October 11, 2026

Task ID/Title: KR-2 — Resume Section Implementation and Testing

Purpose of AI Use:

Frontend development and problem-solving. AI was used to help implement the Resume Section for CareerNet using React and TypeScript. The purpose was to understand how to structure the resume-related files, implement mock API functions, validate uploaded documents, and create a user interface for managing resumes.

Prompt / Request:

Asked ChatGPT to guide me step by step through KR-2, including creating the resume data type, file validation rules, mock resume API functions, Resume Section component, and integration with the existing Profile Page.

AI Response:

ChatGPT provided step-by-step guidance for implementing the Resume Section. It suggested separating the data structure, validation rules, mock API logic, and user interface into different files.

It explained how to validate file types and sizes, temporarily store uploaded files using a JavaScript Map, and implement upload, replace, download, and delete functionality.

ChatGPT also explained how to integrate the Resume Section into the existing Profile Page and suggested manual testing scenarios.

AI-Suggested Content:

The AI suggested creating and modifying the following files:

- src/types/resume.ts — Defines the resume data structure.
- src/constants/resumeRules.ts — Defines allowed file types and the maximum file size.
- src/api/resumes.ts — Provides mock API functions for resume management.
- src/components/ResumeSection.tsx — Implements the Resume Section user interface.
- src/pages/ProfilePage.tsx — Displays the Resume Section within the Profile Page.

The AI also suggested:

- Allowing PDF, DOC, and DOCX documents up to 5 MB.
- Displaying resume filenames, file sizes, and upload dates.
- Implementing upload, replace, download, and delete operations.
- Requiring confirmation before deleting a resume.
- Displaying validation errors and loading messages.
- Testing the functionality with valid and invalid documents.

Validation:

Reviewed and implemented the suggested code in VS Code.

Ran the CareerNet frontend and manually tested the Resume Section, including uploading valid documents, rejecting invalid files, replacing and downloading resumes, and deleting resumes with confirmation.

Confirmed that the Resume Section displays correctly on the Profile Page and that the mock API functions work as expected.

Also reviewed the user interface and improved the Delete button by changing its text color to white for better readability.

The changes were committed and pushed to the feature/KR-2-resume-section branch on GitHub.

Decision:

Accepted and modified before use. I used the AI-generated implementation guidance and made a user-interface adjustment to improve the Delete button's readability. The functionality was manually tested before considering the task ready for review.

Reflection:

AI assistance helped me understand how file uploads and resume management can be implemented in React and TypeScript.

I learned how to validate selected files before uploading, organize API functions separately from React components, and use temporary storage to test frontend features independently from the backend.

I also learned the importance of testing different scenarios, including invalid files, replacement, and delete confirmation.

The AI suggestions provided a starting point, but manually testing the functionality and reviewing the interface were necessary to ensure that the Resume Section worked correctly.

Responsible Person: Krish Patel

AI Usage Log Entry 6 — October 11, 2026

Task ID/Title: KR-3 — Apply Dialog Implementation and Testing

Purpose of AI Use:

Frontend development, debugging, and testing guidance. AI was used to help develop a reusable Apply Dialog component for CareerNet using React and TypeScript.

Prompt / Request:

Asked ChatGPT to guide me step by step through implementing KR-3, including the application API function, resume selection, cover note validation, application confirmation, and error handling.

Also asked for help fixing a TypeScript error involving public status: number in the ApplicationError class.

AI Response:

ChatGPT provided implementation guidance for creating a mock application submission function and a reusable Apply Dialog component.

It suggested using the existing resume API to load uploaded resumes, validating application inputs, preventing duplicate applications, and displaying confirmation or error messages.

For the TypeScript error, ChatGPT suggested moving the status declaration outside the constructor parameter list and assigning its value inside the constructor.

AI-Suggested Content:

The AI suggested:

- Creating src/api/applications.ts with the applyToJob() function.
- Creating src/components/ApplyDialog.tsx.
- Loading resume information using getResumes().
- Adding a resume dropdown and optional cover note.
- Limiting the cover note to 1,000 characters.
- Handling duplicate applications with a 409-style error.
- Showing a confirmation message after successful submission.
- Providing a link to the My Applications page.
- Supporting Escape-key closing and keyboard focus.
- Temporarily adding the component to the Profile Page for manual testing.

Validation:

Reviewed and implemented the suggested code in VS Code.

Ran npm run build successfully after fixing the TypeScript error.

Manually tested resume selection, successful application submission, duplicate application handling, missing resumes, cover note limits, closing the dialog, and keyboard interactions.

Removed the temporary testing code after confirming that the functionality worked.

Verified that the two intended KR-3 files were committed and pushed to GitHub.

Decision:

Accepted and modified before use. Used the AI-generated guidance to implement KR-3 and applied the suggested TypeScript fix after encountering a compilation error. Verified the functionality through build checks and manual testing.

Reflection:

AI assistance helped me understand how to create a reusable React dialog component and connect it to mock application data.

I learned how to validate form inputs, handle duplicate submissions, display success and error states, and improve the accessibility of a dialog using keyboard interactions.

I also learned how to troubleshoot a TypeScript configuration issue and test a component independently before integrating it into another developer's page.

Responsible Person: Krish Patel

AI Usage Log Entry 7 — October 11, 2026

Task ID/Title: KR-4 — Recruiter Dashboard Implementation and Testing

Purpose of AI Use:

Frontend development, Git workflow guidance, and testing support. AI was used to help implement the Recruiter Dashboard for CareerNet using React and TypeScript.

The goal was to create an interface where recruiters can post jobs and view applications, while using mock data until the real backend endpoints become available.

Prompt / Request:

Asked ChatGPT whether KR-4 had any dependencies and requested step-by-step guidance for creating the Recruiter Dashboard.

Also requested help with setting up the correct Git branch, implementing mock recruiter API functions, creating the dashboard interface, adding role-based access, and testing the completed functionality.

AI Response:

ChatGPT reviewed the KR-4 requirements and the GitHub API cheat sheet.

It explained that KR-4 could be developed independently using mock data and recommended creating the feature branch from the latest main instead of the previous KR-3 branch.

It then provided guidance for implementing the recruiter API functions, job creation form, form validation, applications display, protected route, and recruiter-specific navigation link.

ChatGPT also suggested a temporary mock recruiter login for testing and explained how to remove that modification before committing.

AI-Suggested Content:

The AI suggested creating and modifying the following files:

- src/api/recruiter.ts — Mock job posting and application retrieval functions.
- src/pages/RecruiterDashboardPage.tsx — Recruiter Dashboard interface.
- src/App.tsx — Protected recruiter route.
- src/components/Navbar.tsx — Recruiter navigation link.
- src/components/ProtectedRoute.tsx — Role-based access checking.

Additional suggestions included:

- Validating all required job posting fields.
- Using the six agreed job categories.
- Displaying the created job and its status.
- Adding a View Applications button.
- Showing an empty-state message when no applications exist.
- Testing recruiter and job seeker access separately.
- Removing temporary mock-login modifications before committing.

Validation:

Reviewed and implemented the suggested changes in VS Code.

Ran npm run build successfully.

Manually tested creating job postings, required-field validation, displaying job details, viewing the empty applications list, and restricting frontend access based on the logged-in user's role.

Used a temporary mock recruiter login for testing, then restored the original authentication behavior.

Checked the staged Git files and confirmed that only the five intended KR-4 files were included.

Committed the changes and successfully pushed the feature branch to GitHub.

Decision:

Accepted and modified before use. Used AI assistance to implement the Recruiter Dashboard and applied the suggested temporary mock authentication method for testing.

The temporary testing modification was removed before committing. The final implementation was validated through manual testing and a successful production build.

Reflection:

AI assistance helped me understand how to build a recruiter interface using React and TypeScript while separating mock API logic from UI components.

I learned how to validate required form fields, display API results, and implement frontend role-based access.

I also learned the importance of creating independent feature branches when tasks do not depend on one another.

Testing with mock recruiter and job seeker accounts helped me understand how different user roles affect navigation and page access. I understand that the backend must independently enforce authorization when the real APIs are integrated.

Responsible Person: Krish Patel